Programming Principles I

Week 5 · Class 10

Search, min-max and bubble sort

Find a value · the smallest and the largest · put an array in order

```
int temp = a[j];
a[j] = a[j + 1];
a[j + 1] = temp;
```

What Tuesday gave us

1. An array: many values of one type under **one name**
2. `int a[5] = {3, 1, 4, 1, 5};`
3. The first element is `a[0]`, the last is `a[4]`
4. A `for` loop from `0` to `< N` visits every element
5. Outside `0 .. N-1`: out of bounds, no error, wrong values

Recap: what is on the screen?

PREDICT

An array and a loop.

N is 5.

What are the two lines?

```
int main(void) {
    int a[N] = {3, 1, 4, 1, 5};
    printf("%d\n", a[0] + a[4]);

    int sum = 0;
    for (int i = 0; i < N; i++) {
        sum += a[i];
    }
    printf("sum = %d\n", sum);
    return 0;
}
```

Recap: what is on the screen?

PREDICT

An array and a loop.

N is 5.

What are the two lines?

```c
int main(void) {
    int a[N] = {3, 1, 4, 1, 5};
    printf("%d\n", a[0] + a[4]);

    int sum = 0;
    for (int i = 0; i < N; i++) {
        sum += a[i];
    }
    printf("sum = %d\n", sum);
    return 0;
}
```

ANSWER

8, then `sum = 14`

`a[0]` is 3, `a[4]` is 5.

Today's roadmap

1. Arrays and **functions**
2. **Search**: is it there, and where?
3. The **smallest** and the **largest**
4. **Swap** and **bubble sort**

> Four short parts. Each one is a loop over an array, like Tuesday's.

1 Arrays and functions

Send a whole array to a function

A function that prints an array

```
void print_array(int a[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}
```

`int a[]`: empty brackets. Any size.

`int n`: how many elements.

The call: the name, and the size

```
    int primes[5] = {2, 3, 5, 7, 11};
    int ages[3] = {18, 19, 21};

    print_array(primes, 5);
    print_array(ages, 3);
    return 0;
}
```

Output

```
2 3 5 7 11
18 19 21
```

The array's **name**, no brackets.

Then the size: 5, then 3.

Why send the size?

An array does not know how long it is.

Inside the function, `a` is just "an array of `int`".

So the size goes in as a second argument: `n`.

One function, any array: 5 elements, 3 elements, 1000.

A function that returns the sum

```
int sum_array(int a[], int n) {
    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += a[i];
    }
    return sum;
}
```

A loop inside.

One `int` back: the sum.

Using sum_array

```
int marks[N] = {7, 9, 6, 10, 8};
int total = sum_array(marks, N);
printf("sum = %d\n", total);
printf("average = %.2f\n", (double) total / N);
return 0;
```

Output

```
sum = 40
average = 8.00
```

Predict: does the array change?

```
void double_all(int a[], int n) {
    for (int i = 0; i < n; i++) {
        a[i] = a[i] * 2;
    }
}

int main(void) {
    int a[3] = {1, 2, 3};
    printf("before: %d %d %d\n", a[0], a[1], a[2]);
    double_all(a, 3);
    printf("after:  %d %d %d\n", a[0], a[1], a[2]);
    return 0;
}
```

Class 8: `double_it(x)` did **not** change `x`.

What are the two lines?

Run it: the array DID change

```c
void double_all(int a[], int n) {
    for (int i = 0; i < n; i++) {
        a[i] = a[i] * 2;
    }
}

int main(void) {
    int a[3] = {1, 2, 3};
    printf("before: %d %d %d\n", a[0], a[1], a[2]);
    double_all(a, 3);
    printf("after:  %d %d %d\n", a[0], a[1], a[2]);
    return 0;
}
```

Output

```
before: 1 2 3
after:  2 4 6
```

The function changed `main`'s elements.

An int and an array

class 8: an int

```
void double_it(int n) {
    n = n * 2;
    printf("inside: n = %d\n", n);
}

double_it(x);
// x is still 5
```

today: an array

```
void double_all(int a[], int n) {
    for (int i = 0; i < n; i++) {
        a[i] = a[i] * 2;
    }
}

double_all(a, 3);
// a is now 2 4 6
```

An `int` is **copied**. An array is **not** copied: the function works on the caller's elements.

Arrays are not copied

A function gets the caller's own array.

Change `a[i]` inside: `main` sees the change.

Why? Pointers explain it, after the midterm.

Today it is useful: a function can **sort** your array.

2 Search

Is the value there? Where?

Linear search

> **Look at every element, one by one, from the left.**

Equal to the target? **Found**: remember the index, stop.

Reached the end? **Not found**.

> Like looking for your friend in a queue: start at the front, check each face.

Search for 23, step by step

```
int a[N] = {4, 8, 15, 16, 23, 42};
int target = 23;
int pos = -1;

for (int i = 0; i < N; i++) {
    if (a[i] == target) {
        pos = i;
        break;
    }
}
```

i	a[i]	== 23?	pos
0	4	no	-1

pos starts at -1: "not found yet". -1 is never a real index.

Search for 23, step by step

```
int a[N] = {4, 8, 15, 16, 23, 42};
int target = 23;
int pos = -1;

for (int i = 0; i < N; i++) {
    if (a[i] == target) {
        pos = i;
        break;
    }
}
```

i	a[i]	== 23?	pos
0	4	no	-1
1	8	no	-1
2	15	no	-1
3	**16**	**no**	**-1**

4, 8, 15, 16: not 23. The loop goes on.

Search for 23, step by step

```
int a[N] = {4, 8, 15, 16, 23, 42};
int target = 23;
int pos = -1;

for (int i = 0; i < N; i++) {
    if (a[i] == target) {
        pos = i;
        break;
    }
}
```

i	a[i]	== 23?	pos
0	4	no	-1
1	8	no	-1
2	15	no	-1
3	16	no	-1
4	**23**	**yes**	**4**

a[4] is 23. pos = 4, then break: no need to look at 42.

After the loop: found or not?

```
for (int i = 0; i < N; i++) {
    if (a[i] == target) {
        pos = i;
        break;
    }
}

if (pos == -1) {
    printf("%d not found\n", target);
} else {
    printf("%d found at index %d\n", target, pos);
}
```

Output

```
23 found at index 4
```

pos is 4, not -1: found.

Predict: a value that is not there

```
for (int i = 0; i < N; i++) {
    if (a[i] == target) {
        pos = i;
        break;
    }
}

if (pos == -1) {
    printf("%d not found\n", target);
} else {
    printf("%d found at index %d\n", target, pos);
}
```

`target` is now 7.

What is printed?

Predict: a value that is not there

```
for (int i = 0; i < N; i++) {
    if (a[i] == target) {
        pos = i;
        break;
    }
}

if (pos == -1) {
    printf("%d not found\n", target);
} else {
    printf("%d found at index %d\n", target, pos);
}
```

Output

```
7 not found
```

The loop checks all six.

pos stays -1.

Search as a function

```
int find(int a[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (a[i] == target) {
            return i;
        }
    }
    return -1;
}
```

Output

```
find 16: 3
find 7: -1
```

Found: `return i;` ends the function at once.

`return -1;` only **after** the loop.

Careful: one small else

CAREFUL

This `find` has an `else`.

The array is
`4 8 15 16 23 42`.

What does
`find(a, 6, 16)` return?

```
int find(int a[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (a[i] == target) {
            return i;
        } else {
            return -1;
        }
    }
}
```

find 16 says -1

```
int find(int a[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (a[i] == target) {
            return i;
        } else {
            return -1;
        }
    }
}
```

Output

```
find 4: 0
find 16: -1
```

a[0] is 4, not 16: return -1 at once.

16 was never looked at.

gcc saw something too

```
int find(int a[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (a[i] == target) {
            return i;
        } else {
            return -1;
        }
    }
}
```

gcc -Wall says

```
warning: control
reaches end of
non-void function
```

> "Not found" is only true **after** the loop has seen every element. Delete the `else`; put `return -1;` after the loop.

Predict: count the sixes

PREDICT

The rolls are

3 6 2 6 1 6 4 2 5 6.

How many sixes?

How many twos?

```
int count(int a[], int n, int value) {
    int c = 0;
    for (int i = 0; i < n; i++) {
        if (a[i] == value) {
            c++;
        }
    }
    return c;
}
```

Counting: no break

```
int count(int a[], int n, int value) {
    int c = 0;
    for (int i = 0; i < n; i++) {
        if (a[i] == value) {
            c++;
        }
    }
    return c;
}
```

Output

```
sixes: 4
twos: 2
```

Count **all** of them: the loop never stops early.

Search or count?

Search

Is it there? **Where**?

Stop at the first one.

`return i;` or -1

Count

How many?

Look at all of them.

`c++` inside the `if`

3 Smallest and largest

The best so far

The best so far

Start with the first element as the best.

Look at each of the others.

Better than the best? It is the new best.

At the end, the best so far is the best of all.

Like a high-jump contest: the first jump sets the record; each better jump replaces it.

Largest of 12, 45, 7, 30, 19

```
int a[N] = {12, 45, 7, 30, 19};
int max = a[0];
for (int i = 1; i < N; i++) {
    if (a[i] > max) {
        max = a[i];
    }
}
printf("max = %d\n", max);
```

i	a[i]	> max?	max
start			**12**

`max` starts as `a[0]`, 12. So the loop starts at `i = 1`.

Largest of 12, 45, 7, 30, 19

```
int a[N] = {12, 45, 7, 30, 19};
int max = a[0];
for (int i = 1; i < N; i++) {
    if (a[i] > max) {
        max = a[i];
    }
}
printf("max = %d\n", max);
```

i	a[i]	> max?	max
start			12
1	**45**	**yes**	**45**

45 is bigger than 12: the new best.

Largest of 12, 45, 7, 30, 19

```
int a[N] = {12, 45, 7, 30, 19};
int max = a[0];
for (int i = 1; i < N; i++) {
    if (a[i] > max) {
        max = a[i];
    }
}
printf("max = %d\n", max);
```

i	a[i]	> max?	max
start			12
1	45	yes	45
2	7	no	45
3	30	no	45
4	**19**	**no**	**45**

7, 30, 19: none beats 45. It prints `max = 45`.

Careful: start from 0?

CAREFUL

Winter temperatures, all below zero.

`max` starts at `0`.

What is printed?

```
int temps[N] = {-5, -2, -9, -4, -7};
int max = 0;
for (int i = 0; i < N; i++) {
    if (temps[i] > max) {
        max = temps[i];
    }
}
printf("warmest: %d\n", max);
```

Start from 0: wrong

```
int temps[N] = {-5, -2, -9, -4, -7};
int max = 0;
for (int i = 0; i < N; i++) {
    if (temps[i] > max) {
        max = temps[i];
    }
}
printf("warmest: %d\n", max);
```

Output

```
warmest: 0
```

0 is not in the array.

No element beats 0, so `max` never changes.

The fix: `int max = temps[0];` It prints `warmest: -2`. gcc gives no warning here.

The rule

> **Start the best so far at `a[0]`, never at a number you chose.**

`0` fails when all values are negative.

`100` fails when all values are above 100.

`a[0]` is always a real value from the array.

Smallest: flip one sign

```
int min_of(int a[], int n) {
    int min = a[0];
    for (int i = 1; i < n; i++) {
        if (a[i] < min) {
            min = a[i];
        }
    }
    return min;
}
```

< instead of >.

Same start: a[0].

min_of and max_of in main

```
int main(void) {
    int temps[N] = {14, 17, 11, 19, 16, 9, 13};
    printf("coldest: %d\n", min_of(temps, N));
    printf("warmest: %d\n", max_of(temps, N));
    return 0;
}
```

Output

```
coldest: 9
warmest: 19
```

Two functions, one array.

Where is the smallest?

```
int price[N] = {12, 9, 7, 11, 8};
int pos = 0;
for (int i = 1; i < N; i++) {
    if (price[i] < price[pos]) {
        pos = i;
    }
}
```

i	price[i]	< price[pos]?	pos
start			**0**

Keep the **index** of the best, not the value: `pos`. Its value is `price[pos]`.

Where is the smallest?

```
int price[N] = {12, 9, 7, 11, 8};
int pos = 0;
for (int i = 1; i < N; i++) {
    if (price[i] < price[pos]) {
        pos = i;
    }
}
```

i	price[i]	< price[pos]?	pos
start			0
1	9	yes	1
2	7	yes	2
3	11	no	2
4	**8**	**no**	**2**

It prints `cheapest: index 2, price 7`. With the index you also have the value.

4

Swap and bubble sort

Put the array in order

Careful: swap two elements

CAREFUL

`a[0]` is 3, `a[1]` is 7.

We want `7 3`.

What is printed?

```c
#include <stdio.h>

int main(void) {
    int a[2] = {3, 7};
    a[0] = a[1];
    a[1] = a[0];
    printf("%d %d\n", a[0], a[1]);
    return 0;
}
```

Both are 7

```c
#include <stdio.h>

int main(void) {
    int a[2] = {3, 7};
    a[0] = a[1];
    a[1] = a[0];
    printf("%d %d\n", a[0], a[1]);
    return 0;
}
```

Output

```
7 7
```

Line 5: `a[0]` becomes 7. The 3 is **lost**.

Line 6 copies that 7 back.

Swap with a third box

```c
#include <stdio.h>

int main(void) {
    int a[2] = {3, 7};
    int temp = a[0];
    a[0] = a[1];
    a[1] = temp;
    printf("%d %d\n", a[0], a[1]);
    return 0;
}
```

Output

```
7 3
```

`temp` keeps the 3 safe.

A swap in three moves

1. save

```
temp = a[0];
```

temp: 3

2. copy

```
a[0] = a[1];
```

a[0]: 7

3. restore

```
a[1] = temp;
```

a[1]: 3

> Like swapping the drinks in two full glasses: you need a third, empty glass.

Bubble sort: the idea

> **Compare two neighbours. Wrong order? Swap them.**

Do it for every pair, from the left: that is one **pass**.

After one pass, the **largest** is at the end.

Repeat the pass until the array is in order.

The big numbers rise to the end like bubbles in water.

One pass over 5 3 8 4 2

compare	swap?	array after
5 and 3	yes	3 5 8 4 2

Start at the left: 5 and 3 are in the wrong order. Swap.

One pass over 5 3 8 4 2

compare	swap?	array after
5 and 3	yes	**3 5** 8 4 2
5 and 8	**no**	**3 5 8 4 2**

5 and 8 are in order. Leave them.

One pass over 5 3 8 4 2

compare	swap?	array after
5 and 3	yes	**3 5** 8 4 2
5 and 8	no	3 **5 8** 4 2
8 and 4	yes	3 5 **4 8** 2
8 and 2	**yes**	**3 5 4 2 8**

The 8 is carried to the end, swap by swap. It is now in its place.

One pass in C

```
int a[N] = {5, 3, 8, 4, 2};
for (int j = 0; j < N - 1; j++) {
    if (a[j] > a[j + 1]) {
        int temp = a[j];
        a[j] = a[j + 1];
        a[j + 1] = temp;
    }
}
for (int i = 0; i < N; i++) {
    printf("%d ", a[i]);
}
printf("\n");
```

Output

```
3 5 4 2 8
```

`j < N - 1`: `a[j + 1]` stays inside.

Pass after pass

pass	array after it	in place
start	5 3 8 4 2	
1	**3 5 4 2 8**	**8**

After pass 1 the largest, 8, is in its place.

Pass after pass

pass	array after it	in place
start	5 3 8 4 2	
1	3 5 4 2 **8**	8
2	**3 4 2 5 8**	**5 8**

Pass 2 does the same for the rest: 5 goes next to the 8.

Pass after pass

pass	array after it	in place
start	5 3 8 4 2	
1	3 5 4 2 **8**	8
2	3 4 2 **5 8**	5 8
3	3 2 **4 5 8**	4 5 8
4	**2 3 4 5 8**	**all**

5 numbers, 4 passes. After the last pass the smallest, 2, is first by itself.

Bubble sort: two loops

```
int a[N] = {5, 3, 8, 4, 2};
for (int pass = 1; pass < N; pass++) {
    for (int j = 0; j < N - pass; j++) {
        if (a[j] > a[j + 1]) {
            int temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
        }
    }
    printf("pass %d: ", pass);
    print_array(a, N);
}
```

Outer: the passes.

Inner: the pairs.

Why j < N - pass?

```
int a[N] = {5, 3, 8, 4, 2};
for (int pass = 1; pass < N; pass++) {
    for (int j = 0; j < N - pass; j++) {
        if (a[j] > a[j + 1]) {
            int temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
        }
    }
    printf("pass %d: ", pass);
    print_array(a, N);
}
```

After pass 1 the last element is in place.

Each pass can stop one pair earlier.

Run it: watch every pass

```
int a[N] = {5, 3, 8, 4, 2};
for (int pass = 1; pass < N; pass++) {
    for (int j = 0; j < N - pass; j++) {
        if (a[j] > a[j + 1]) {
            int temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
        }
    }
    printf("pass %d: ", pass);
    print_array(a, N);
}
```

Output

```
pass 1: 3 5 4 2 8
pass 2: 3 4 2 5 8
pass 3: 3 2 4 5 8
pass 4: 2 3 4 5 8
```

Predict: four numbers

PREDICT

Bubble sort on `4 3 2 1`.

How many passes?

What is the array after pass 1?

Predict: four numbers

PREDICT

Bubble sort on `4 3 2 1`.

How many passes?

What is the array after pass 1?

ANSWER

3 passes (4 numbers, 4 - 1).

After pass 1: `3 2 1 4`. The 4 is in place.

Sort as a function

```
void bubble_sort(int a[], int n) {
    for (int pass = 1; pass < n; pass++) {
        for (int j = 0; j < n - pass; j++) {
            if (a[j] > a[j + 1]) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}
```

`void`: it returns nothing.

It changes the array.

Sorting eight marks

```
int marks[N] = {72, 95, 61, 88, 79, 54, 90, 67};
print_array(marks, N);
bubble_sort(marks, N);
print_array(marks, N);
return 0;
```

Output

```
72 95 61 88 79 54 90 67
54 61 67 72 79 88 90 95
```

`main`'s array is now sorted.

Extra: stop when nothing moves

```
int swapped = 0;
for (int j = 0; j < N - pass; j++) {
    if (a[j] > a[j + 1]) {
        int temp = a[j];
        a[j] = a[j + 1];
        a[j + 1] = temp;
        swapped = 1;
    }
}
passes++;
if (!swapped) {
    break;
}
```

Output for 1 2 3 5 4

```
passes: 2
1 2 3 4 5
```

A pass with no swap: sorted.

Sorting high to low

Change one character: `>` becomes `<`.

`if (a[j] < a[j + 1])`: the **smaller** goes to the end.

Everything else stays the same.

5

Your turn

Two small functions

Your turn: index_of_max

YOUR TURN

Write `int index_of_max(int a[], int n)`.

It returns the **index** of the largest element.

For goals `1 0 3 4 2 1` it returns 3.

index_of_max: one answer

```
int index_of_max(int a[], int n) {
    int pos = 0;
    for (int i = 1; i < n; i++) {
        if (a[i] > a[pos]) {
            pos = i;
        }
    }
    return pos;
}
```

Output

```
best match: index 3
```

Like the cheapest shop, with >.

Your turn: contains

YOUR TURN

Write `int contains(int a[], int n, int x)`.

It returns 1 if `x` is in the array, 0 if not.

Where does the `return 0;` go?

contains: one answer

```
int contains(int a[], int n, int x) {
    for (int i = 0; i < n; i++) {
        if (a[i] == x) {
            return 1;
        }
    }
    return 0;
}
```

Run, input 13

```
Number: 13
yes, lucky
```

`return 0;` **after** the loop.

Today in one slide

1. Send an array **and its size**: `f(a, n)`
2. A function can **change** the caller's array
3. Search: return the index, or `-1` **after** the loop
4. Min and max: start from `a[0]`
5. Swap with `temp`; bubble sort: passes of neighbour swaps

Before Tuesday

1. **Knowledge check 10** on Blackboard: today's slides
2. **Project 1** (Blackjack): due **Sun 11 October, 23:59**
3. **Quiz 3: Fri 16 October**, classes 7 to 10
4. Today's code: Blackboard, Week 5 module, **Class 10 code**

> Next class: characters and strings. A string is an array of `char`.